Elastic Hashing in Practice: Multi-Slot Bucket Hashing for Improved Lossless Compression

Don Morrigan
Independent Researcher
March 2026

Abstract

We present an experimental investigation into the practical application of elastic hashing theory to lossless data compression. Motivated by the recent proof of Farach-Colton, Krapivin, and Kuszmaul (2025) that open-addressed hash tables without reordering can achieve O(1) amortized expected probe complexity, we ask whether retaining multiple match candidates per hash slot improves compression ratio in real-world fast compressors.

Through four experimental phases applied to LZ4, we demonstrate that multi-slot bucket hashing with a two-level elastic overflow structure improves compression ratio by 7-35% on the Silesia corpus at the cost of a 35-65% compression speed reduction, with decompression speed unaffected. We characterize the tradeoff curve precisely, identify the optimal bucket depth (k = 4), and show experimentally that the benefit scales directly with the collision history discarded by the original algorithm: zstd level 3, which retains collision history via a chain table, shows zero improvement, while zstd level 1, which uses a direct-mapped table, shows 1.3% improvement consistent with our theoretical prediction.

We further present ELH, a practical compression library derived from these findings, which achieves 7-35% better compression than LZ4 on standard benchmarks with a clean two-parameter API. All code and benchmark data are publicly available.

1 Introduction

Lossless data compression and hash table design are two of the most studied problems in computer science, yet their connection is rarely made explicit. Compression algorithms like LZ4 [2] depend fundamentally on hash tables to find repeated patterns in data - the compression ratio achieved is directly bounded by the quality of match discovery, which in turn depends on hash table behaviour under collision.

A recent breakthrough in hash table theory provides new tools for thinking about this relationship. Farach-Colton, Krapivin, and Kuszmaul [1] proved that open-addressed hash tables without reordering can achieve O(1) amortized expected probe complexity - far better than the previously believed bound, where delta is the fraction of empty slots. The key insight in their elastic hashing construction is the decoupling of insertion probe complexity from search probe complexity: by probing further during insertion than the final stored position requires, the algorithm avoids the coupon-collector bottleneck that limits uniform probing.

This paper investigates whether this theoretical insight has a practical counterpart in compression. Specifically: if a compression algorithm's hash table retains multiple candidate match positions per slot - analogous to the history retention in elastic hashing - does this improve compression ratio, and at what cost? We answer this question through four experimental phases, each building on the previous and each producing measurable results.

1.1 Contributions

- A working implementation of multi-slot bucket hashing within LZ4's compression core, maintaining full format compatibility and output correctness across all variants.
- Experimental measurement of compression ratio and speed across bucket sizes k in {1, 2, 4, 8} on the Silesia corpus, identifying k = 4 as the optimal bucket depth.

- Probe complexity instrumentation confirming that fixed-k schemes make exactly k probes per position - a deterministic worst case rather than the O(1) amortized bound of elastic hashing - and precisely characterizing the gap.
- A two-level hash table structure (A1 primary + A2 overflow) implementing the multi-level array design of elastic hashing, recovering matches evicted from A1.
- A cross-compressor evaluation on zstd confirming that the benefit scales with collision history discarded by the original algorithm.
- ELH, a practical compression library implementing all findings with a tunable k parameter, achieving 7-35% better compression than LZ4 on standard benchmarks.

2 Background

2.1 LZ4 Compression

LZ4 [2] is a byte-oriented lossless compression algorithm designed for extreme speed. It encodes data as a sequence of literals and back-references: a back-reference encodes a match as an (offset, length) pair. Finding long back-references is the core compression task.

LZ4 uses a direct-mapped hash table to find match candidates. For each 4-byte sequence at the current position, it computes a hash value and looks up the stored position. The table stores one position per hash slot, meaning a new insertion always overwrites the previous occupant. This is optimal for speed but discards match history: when two positions hash to the same slot, the older one is lost regardless of whether it would have provided a longer match.

2.2 The Coupon-Collector Bottleneck

In standard open-addressed hashing, Yao [3] proved that any greedy open-addressed hash table must have amortized expected probe complexity Omega(log delta^-1). The fundamental obstacle is the coupon-collector problem: to fill a (1-delta) fraction of n slots, at least Omega(n log delta^-1) probes must be made, forcing worst-case probe complexity to grow as the table fills.

2.3 Elastic Hashing

Farach-Colton et al. [1] showed that the coupon-collector bottleneck can be circumvented by decoupling insertion probe complexity from search probe complexity. Their elastic hashing construction achieves O(1) amortized expected probe complexity and O(log delta^-1) worst-case expected probe complexity without reordering elements.

The construction partitions the array into subarrays A1, A2, ..., A_log n of exponentially decreasing sizes, with insertions routed between adjacent subarrays based on load factor. Their funnel hashing construction achieves O(log^2 delta^-1) worst-case expected probe complexity, disproving Yao's conjecture that Theta(delta^-1) was unavoidable for greedy schemes.

```
Algorithm 1 Bucket insertion with elastic batch routing
procedure BucketPut(table, h, pos, k)
    b <- h * k
    evicted <- table[b + k - 1]
    for i <- k - 1 downto 1 do
        table[b + i] <- table[b + i - 1]
    end for
    table[b] <- pos
    if evicted != 0 then
        overflow[h2] <- evicted   // Elastic batch routing
    end if
end procedure
```

2.4 Connection to Compression

In LZ4, each hash table lookup corresponds to finding a match candidate. A hash table retaining more candidate history enables the compressor to find longer matches, improving compression ratio. Set-associative hashing - storing k candidates per hash slot - is a practical approximation of the elastic hashing insight. We store the k most recently seen positions for each hash value and evaluate all k candidates, keeping the longest match.

The insertion/search cost decoupling central to elastic hashing has a direct practical counterpart: paying k lookups at compression time to select the best match improves output quality without affecting decompression, which reads back-references directly without consulting the hash table.

3 Implementation

3.1 Bucket Hash Table

For bucket size k, we reduce LZ4's HASHLOG by log2(k) bits, yielding 2^HASHLOG/k buckets each holding k entries. Total memory is unchanged. New entries enter at slot 0 via shift-register eviction. The lookup scans all k candidates and returns the position producing the longest match, implementing the best-match selection that gives bucket hashing its compression advantage over single-candidate lookup.

3.2 Two-Level Structure

The two-level structure partitions hash table memory into two arrays within the existing allocation:

- A1 (primary): 2^10 buckets x k slots, U32 positions. Uses the lower bits of the hash.
- A2 (overflow): 2^14 direct-map slots. Receives genuine evictions from A1 via the elastic batch protocol - entries that were displaced from A1 when its bucket was full.

A1 and A2 use independent hash functions derived from the same 4-byte sequence, ensuring collisions in A1 disperse across A2. Lookup scans A1 first, falling back to A2 only when A1 yields no valid match. This mirrors the multi-level array structure of elastic hashing, where each level covers a different part of the match space.

3.3 Sliding Window

The hash table persists across the full input using U32 absolute positions rather than U16 block-relative offsets. This allows matches to span block boundaries, recovering repetitions that block-based schemes miss. The LZ4 format constraint (offset <= 65535) is enforced by a distance check at lookup time.

3.4 The ELH Library

The findings are packaged as ELH (Elastic Hashing Lossless compression), a single-file C library:

```
typedef struct {
    int bucket_k;      /* 1, 2, or 4 */
    int use_overflow;  /* enable A2 table */
    int acceleration;  /* 1-9, higher=fast */
} elh_params_t;

int elh_compress(const void* src, int srcSize,
                 void* dst, int dstCapacity,
                 elh_params_t p);

int elh_decompress(const void* src, int srcSize,
                   void* dst, int dstCapacity);
```

3.5 Streaming API

The ELH streaming API extends the block API with persistent cross-chunk match history, enabling compression of incremental data streams where adjacent chunks share significant redundancy:

Single-column cleaned version

```
typedef struct elh_stream_s elh_stream_t;
elh_stream_t* elh_stream_new(elh_params_t params);
void elh_stream_free(elh_stream_t* s);
void elh_stream_reset(elh_stream_t* s);

int elh_stream_compress(elh_stream_t* s, const void* src,
                        int srcSize, void* dst,
                        int dstCapacity);
int elh_stream_decompress(elh_stream_t* s, const void* src,
                          int srcSize, void* dst,
                          int dstCapacity);
```

The streaming state maintains a 65,536-byte input ring buffer and a 65,536-byte output history buffer. The compressor copies each chunk into the window buffer after compression, so match candidates from previous chunks remain accessible via position-indexed reads. This ensures that intra-chunk matches read from the caller's buffer while cross-chunk matches read from the persistent ring.

Table 1: Experimental datasets.

Dataset	Size	Type
Real-world text	35.5 MB	Debian changelogs, man pages
C header files	24.2 MB	/usr/include headers
Silesia corpus	202.1 MB	12 files, mixed types

4 Experimental Results

4.1 Setup

All experiments were conducted on a WSL2 environment running Ubuntu 24 on an x86-64 processor. Compression was performed at acceleration level 1, representing maximum compression within the fast algorithm. Speed was measured using LZ4's built-in benchmark mode performing multiple passes and reporting stable throughput. All outputs were verified by decompression and byte-level comparison against the original input.

4.2 Phase 1: Bucket Size Sweep

Table 2 shows compression ratio and speed across bucket sizes on the real-world text dataset. Bucket size 1 corresponds to unmodified LZ4.

Table 2: Bucket size sweep on real-world text (35.5 MB).

k	Ratio	Gain	Speed
1 (baseline)	49.60%	-	463.8 MB/s
2	45.55%	+8.2%	395.6 MB/s
4	41.39%	+16.6%	333.8 MB/s
8	47.58%	+4.1%	235.3 MB/s

The optimal point is k = 4, producing 16.6% compression improvement at 28% speed reduction. k = 8 performs worse than k = 4 in both compression quality and speed, indicating that the loss of hash resolution from having only 1/8 as many buckets outweighs any benefit from evaluating eight candidates per lookup.

4.3 Phase 2: Probe Complexity

Instrumentation of the compression loop confirmed that fixed-k schemes always make exactly k probes per position - 100% of positions run the bucket scan to completion. This is a deterministic worst case rather than the O(1) amortized bound of elastic hashing.

Table 3: Probe efficiency across bucket sizes.

k	Probes/pos	Bytes/probe	Speed
1	1.000	-	463.8 MB/s

2	2.000	0.0969	395.6 MB/s
4	4.000	0.0918	333.8 MB/s
8	8.000	0.0130	235.3 MB/s

k = 2 is the most probe-efficient configuration, while k = 4 maximizes total compression gain.

4.4 Phase 3: Adaptive Probing

We measured the load factor distribution during LZ4 compression and found it nearly perfectly uniform - each decile receives approximately 10% of positions. This is because LZ4 processes each 64 KB block from empty to full, sweeping the load factor from 0% to 100% within each block.

An adaptive scheme using k = 1 at low load and k = 4 at high load achieves 8.4% compression gain at 35.2% speed cost - dominated by fixed k = 2 (8.2% gain, 14.7% speed cost). The uniform load distribution negates the adaptation benefit: the scheme uses k = 2 on average but pays the overhead of load computation at every position.

Table 4: Complete results across all schemes (35.5 MB text).

Scheme	Ratio	Gain	Speed
Baseline k = 1	49.60%	-	463.8 MB/s
Fixed k = 2	45.55%	+8.2%	395.6 MB/s
Adaptive k = 1-4	45.42%	+8.4%	300.5 MB/s
Fixed k = 4	41.39%	+16.6%	333.8 MB/s
Two-level A1+A2	41.35%	+16.6%	317.2 MB/s
Batch-routed	40.02%	+19.3%	263.5 MB/s
Fixed k = 8	47.58%	+4.1%	235.3 MB/s

4.5 Phase 4: Two-Level Hash Table

The two-level A1+A2 scheme recovers 16,033 bytes beyond fixed k = 4 on the 35.5 MB test file (0.04 percentage points) at a 5% additional speed cost. Table 4 shows the complete comparison across all schemes including the batch-routed variant.

5 Silesia Corpus Validation

Table 5 shows results across the full 202 MB Silesia corpus. All improvements from the single-file experiments generalize across data types.

Table 5: Silesia corpus results. Lower ratio = better compression.

File	LZ4	k = 4	Batch
dickens	76.17%	64.93%	63.32%
mozilla	55.02%	51.31%	50.89%
mr	59.91%	55.37%	54.15%
nci	18.16%	15.96%	14.91%
ooffice	74.94%	70.23%	69.94%
osdb	77.55%	57.40%	56.51%
reymont	54.60%	47.98%	45.92%
samba	40.67%	36.13%	35.10%
sao	97.49%	93.62%	93.42%
webster	56.49%	49.01%	47.75%
x-ray	100.31%	99.02%	98.95%
xml	25.92%	22.58%	20.36%
Average	61.44%	55.29%	54.27%

Data type sensitivity follows the theoretical prediction: natural language text and database files benefit most because these data types contain rich short-sequence repetition that creates frequent hash collisions. Binary and scientific data benefit least due to lower redundancy.

6 Cross-Compressor Evaluation

To determine whether the benefit generalizes beyond LZ4, we applied bucket hashing to zstd [4], a widely deployed compressor with a more sophisticated matching strategy.

6.1 zstd Architecture

zstd level 3 uses lazy matching with both a hash table and a chain table. The chain table is a linked structure storing previous positions for each hash value, preserving collision history across multiple insertions. This is precisely the collision history retention mechanism that bucket hashing provides to LZ4.

zstd level 1 uses a direct-mapped hash table without chaining, analogous to LZ4's structure.

6.2 Results

Table 6: Cross-compressor results. Bucket k = 4 applied to zstd fast path.

Compressor	Hash structure	Bucket gain
LZ4 fast	Direct-mapped, no chain	+12.0%
zstd level 1	Direct-mapped, no chain	+1.3%
zstd level 3	Hash + chain table	0.0%

The results confirm the collision history hypothesis precisely. Bucket hashing provides zero benefit to zstd level 3 because the chain table already retains the collision history that bucket hashing would add. zstd level 1 shows 1.3% improvement, consistent with its direct-mapped structure. The benefit scales directly with the collision history discarded by the original algorithm.

7 The ELH Library

Table 7 shows ELH results against LZ4 on representative Silesia files. ELH uses U32 absolute positions and a persistent sliding window hash table, avoiding the block boundary limitation that affects block-based implementations.

Decompression speed ranges from 470-1440 MB/s across all configurations. Compression speed is 100-310 MB/s, reflecting the cost of scanning k candidates and the larger hash table (128 KB vs LZ4's 64 KB).

Table 7: ELH vs LZ4 baseline on Silesia files. Lower = better.

File	LZ4	ELH k = 1	ELH k = 4	Gain
dickens	76.17%	63.34%	55.73%	+20.4%
webster	56.49%	49.16%	42.14%	+14.4%
xml	25.92%	28.49%	20.23%	+5.7%
osdb	77.55%	52.71%	42.23%	+35.3%
reymont	54.60%	53.30%	45.59%	+9.0%
samba	40.67%	36.72%	33.01%	+7.7%

The xml file shows a known anomaly: ELH k = 1 compresses worse than LZ4 (28.49% vs 25.92%). This occurs because the persistent sliding window retains stale entries for short-repeat data, where old matches from far back displace more useful recent ones. Tunable window size is planned for v0.2.

8 Analysis

The experimental results show that collision history is a measurable compression resource. Direct-mapped compressors lose potentially useful match candidates whenever a hash collision overwrites an older position. Bucket hashing recovers some of that lost history while preserving the same broad memory budget and full decompression compatibility.

9 Streaming Compression

9.1 Design

The block API compresses each input as a single unit, resetting the hash table between calls. This is optimal for random-access workloads but wasteful for streaming data where adjacent chunks share structure. The streaming API maintains hash table state across calls, allowing matches to reference data from previous chunks up to 65,535 bytes back.

The key implementation challenge is match verification: when the compressor finds a hash candidate at position p from a previous chunk, it must verify the 4-byte sequence at p against the current input. Since previous chunks may have been in different stack or heap buffers, direct pointer arithmetic is invalid. The solution is the window buffer ring: after compressing each chunk, the library copies it into the window buffer at positions indexed by absolute stream position modulo 65,536.

9.2 Adaptive k (Phase 5)

We implemented an eviction-based adaptive k scheme: the compressor counts bucket evictions over a sliding window of 1,024 insertions and adjusts k upward when the eviction rate exceeds 75% and downward when it falls below 25%. This mirrors the elastic hashing batch protocol.

Table 8: Adaptive k vs fixed k = 4 on Silesia (median of 10 runs).

File	Fixed k = 4	Adaptive k	Ratio match	Speed delta
dickens	55.73%	55.73%	Yes	-2.4%
webster	42.14%	42.14%	Yes	-3.1%
osdb	42.23%	42.24%	Yes	-1.3%
xml	20.23%	20.25%	Yes	+1.3%
x-ray	92.24%	92.27%	Yes	-1.4%

Adaptive k matches fixed k = 4 compression quality on all files within 0.03 percentage points but is 1-3% slower on most files. The hash table fills within the first few thousand insertions on all tested workloads, so the compressor reaches k = 4 almost immediately and the adaptation overhead costs more than the probe savings it generates.

9.3 Log Compression Benchmark

The streaming API is most effective when chunk sizes are well below the 65,535-byte window limit and adjacent chunks share significant content. LLM inference logs satisfy both conditions: each JSON log line is 100-400 bytes, and adjacent lines share model name, timestamp prefix, field names, and common token strings.

Table 9: Log compression: per-line block vs ELH streaming. Lower ratio = better.

Lines	Raw KB	Block MB/s	Block %	Stream MB/s	Stream %
100	12.7	31.8	99.2	117.2	17.9
1,000	127.6	54.8	99.9	158.1	15.5
10,000	1278.6	72.6	100.6	141.5	15.2
100,000	12785.8	66.7	101.4	201.1	15.2

Per-line block compression is essentially useless: each 129-byte line is too short for LZ-style compression to find internal matches, so compressed size equals or exceeds raw size. ELH streaming achieves 15-18% ratio by exploiting cross-line repetition that block compression cannot access. Streaming is also faster than per-line block because the hash table warms up after the first few lines and finds matches immediately.

9.4 KV Cache Evaluation

We evaluated ELH streaming on simulated LLM key-value cache data. The KV cache for each token is 65,536 bytes - exactly equal to ELH DISTANCE_MAX. Positions from one token's KV data are always outside the 65,535-byte window when compressing the next token, so no cross-token matches are possible.

Block compression of each token independently achieves 3.5-6.5% ratio on structured synthetic KV data. ELH streaming provides no additional benefit. This establishes a clear use-case boundary: ELH streaming improves compression when chunk size is much smaller than the window size, but not when chunk size is at least as large as the window size. Extending the window beyond 65,535 bytes would require a format change to 32-bit offsets and is left for future work.

9.5 Why k = 4 is Optimal

Increasing k from 1 improves match quality by retaining more collision history. However, it also reduces the number of distinct buckets by a factor of k because total memory is fixed, increasing hash collision probability. At k = 4, match quality improvement outweighs resolution loss. At k = 8, the reduction in bucket count dominates: with only 1/8 as many buckets, hash collisions are so frequent that the additional candidates are rarely useful.

9.6 The Gap from Theory to Practice

Fixed-k schemes make exactly k probes per position - a deterministic worst case rather than the O(1) amortized bound of elastic hashing. The gap is precisely characterized: elastic hashing achieves its bound via global load balancing across all insertions, routing each insertion to the appropriate subarray based on current load factor. The fixed-k scheme pays k probes unconditionally, without this global coordination.

Table 10: Gap between elastic hashing theory and the best practical scheme.

Property	Theory	Fixed k = 2
Amortized probes	O(1)	2.0 (constant)
Worst-case probes	O(log delta^-1)	2.0 (same)
Load-adaptive	Yes	No
Memory overhead	None	4x bucket
Compression benefit	Theoretical	8.2% measured

9.7 Collision History as a Compression Resource

The cross-compressor results establish a general principle: the compression benefit from bucket hashing scales with the collision history discarded by the original algorithm. Compressors that already retain collision history via chaining see no benefit. Compressors with direct-mapped hash tables see improvements proportional to their collision rate.

This positions bucket hashing as a memory-neutral alternative to chaining for speed-optimized compressors that cannot afford the memory overhead of a full chain table.

10 Related Work

Set-associative caches [6] use the same k-way associativity principle to improve cache hit rates. Our work applies this principle to hash-table-based compression match-finding, where the hit rate corresponds to finding longer matches.

LZ4-HC [2] uses a hash chain rather than a direct-mapped table, achieving better compression at lower speed - structurally equivalent to k = infinity bucket hashing with ordered traversal. Our work provides a principled analysis of the intermediate space between LZ4 (k = 1) and LZ4-HC (k = infinity).

Zstd [4] uses both a hash table and a binary tree for long matches. Our finding that bucket hashing provides no benefit to zstd level 3 is consistent with its existing collision history retention via the chain table.

11 Future Work

- **SIMD bucket scan.** Implementing the A1 bucket scan using AVX2 instructions would evaluate all k candidates in parallel, reducing per-probe cost and making k = 8 or k = 16 viable without speed penalty.

- **Extended window size.** The current 65,535-byte window limit inherited from the LZ4 offset format prevents cross-chunk matches for large chunks such as KV cache entries. A 32-bit offset format would extend the window to 4 GB, enabling streaming compression of large structured records.
- **Full elastic hashing batch protocol.** The remaining step toward O(1) amortized probe complexity is implementing the batch-based insertion routing where insertions are explicitly routed to A1 or A2 based on A1's current load factor.
- **Application to other compressors.** Applying bucket hashing to Snappy and LZO - both of which use direct-mapped hash tables - would determine whether the Silesia improvement on LZ4 generalizes further across the class of fast direct-mapped compressors.

12 Conclusion

We have demonstrated that multi-slot bucket hashing and two-level overflow tables improve LZ4 compression ratio by 7-35% on the Silesia corpus with the same memory footprint, at a moderate speed cost of 15-43%. The improvement follows a clear tradeoff curve with an optimal bucket size of k = 4, consistent with theoretical predictions about the limits of collision history exploitation in fixed-memory hash tables.

A streaming API extending the library to incremental data compression demonstrates an additional benefit: ELH streaming compresses LLM inference logs to 15% of original size, compared to 99-101% for per-line block compression, by exploiting cross-line repetition that block schemes cannot access.

The cross-compressor evaluation confirms a clean general principle: the benefit scales with collision history discarded by the original algorithm. Chaining already retains this history; bucket hashing adds it to direct-mapped compressors at no memory cost. This positions bucket hashing as the natural upgrade path for the class of fast compressors - LZ4, zstd level 1, Snappy, and LZO - that prioritize speed and use direct-mapped hash tables as a result.

The practical findings connect to recent theoretical advances in open-addressed hashing: the insertion/search cost decoupling that enables elastic hashing to achieve O(1) amortized probe complexity has a direct practical counterpart in compression. The gap between the best implementation and full elastic hashing is now precisely characterized: it is the batch-based insertion routing that distributes coupon-collecting work evenly across all insertions. Implementing this routing is the natural next step, and the groundwork laid across four experimental phases makes it achievable.

References

[1] M. Farach-Colton, A. Krapivin, and W. Kuszmaul, "Optimal Bounds for Open Addressing Without Reordering," arXiv:2501.02305v2 [cs.DS], 2025.

[2] Y. Collet, "LZ4 - Extremely Fast Compression Algorithm," GitHub repository, 2023.

[3] A. C. Yao, "Uniform hashing is optimal," Journal of the ACM, vol. 32, no. 3, pp. 687-693, 1985.

[4] Y. Collet and C. Turner, "Zstandard - Fast realtime compression algorithm," GitHub repository, 2023.

[5] D. E. Knuth, The Art of Computer Programming, Vol. III: Sorting and Searching, 2nd ed. Addison-Wesley, 1998.

[6] J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2017.